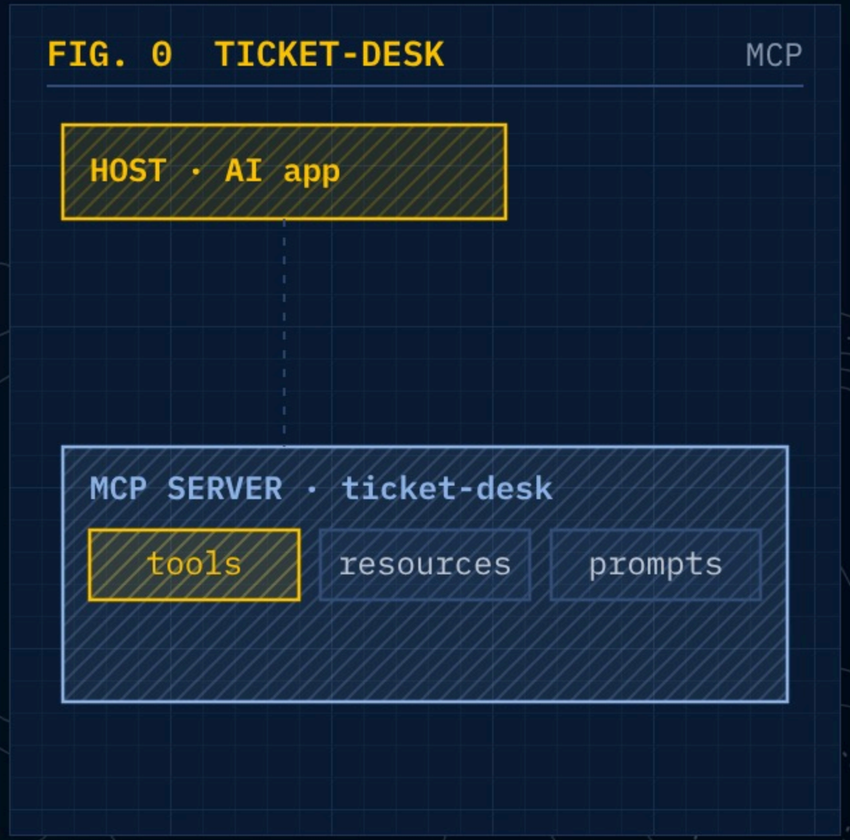
Kinvecta

TRAIN-THE-TRAINER SERIES · DEEP DIVE A

# Build an MCP server from scratch

Few tools, tight schemas, a person before every action, and your own tokens.




20-minute video

Technical · DevSecOps

MCP spec 2026-07-28

Kinvecta

TODAY

# By the end, your learners can…

**1**

Explain host, client and server, and what a request looks like today

**2**

Build a server with a read-only tool, an action that asks first, a resource and a prompt

**3**

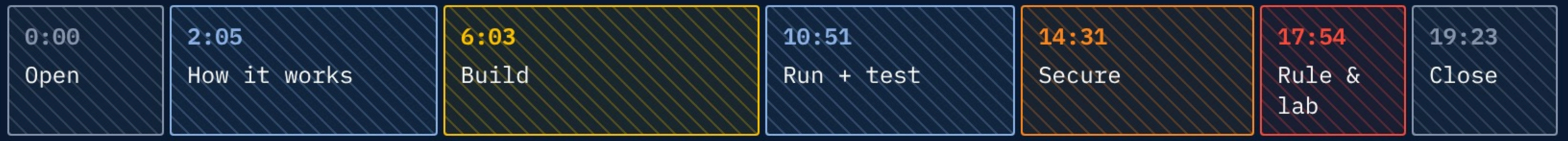
Run it over stdio and HTTP, test it, and connect it to a host

**4**

Secure it: sign-in, no passthrough, threats and controls, release

CHAPTERS · 20 MIN

drawn to scale



Kinvecta

WHY BUILD IT CAREFULLY

# Three reasons to build it right the first time

01

## Tutorials went stale in July 2026

Spec 2026-07-28 dropped the handshake and sessions; both SDKs renamed their APIs in v2.

02

## A server acts with your authority

A model picks the tool, and what it returns goes straight back into the model's context.

03

## The incidents are real

A backdoored npm server BCC'd every email it sent; a public issue made an agent leak private repo data.

Today's question: **How do we build a server that's current today and safe by design?**

THE ARCHITECTURE

# Host, client, server

## Host = the AI app

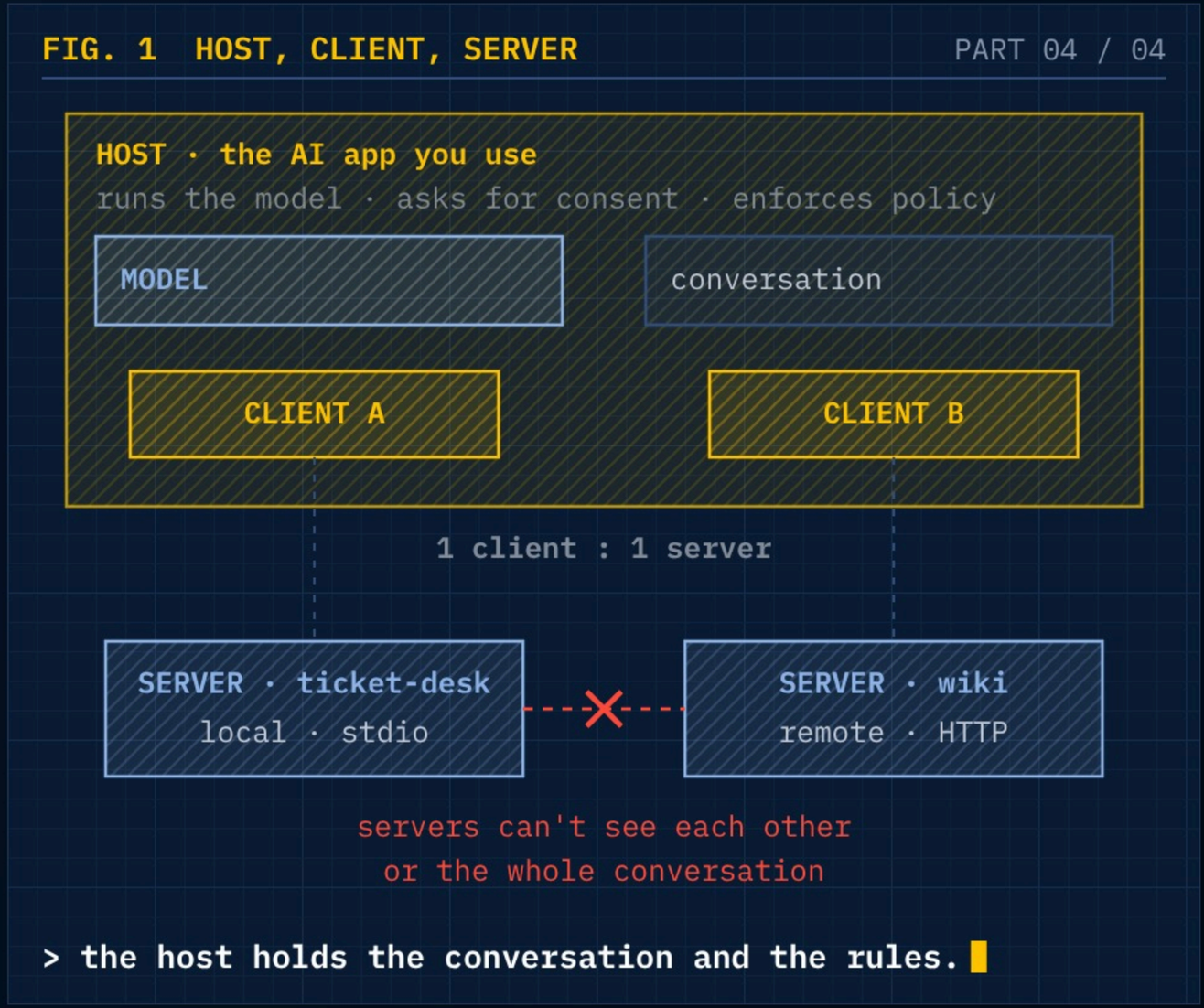
Runs the model, holds the conversation, asks for consent

## Client = one per server

A connector inside the host, paired 1:1 with a server

## Server = your program

Exposes tools, resources and prompts; local or remote



Kinvecta

THE BIG PICTURE

# One request, four layers, three trust boundaries

### The host asks the person

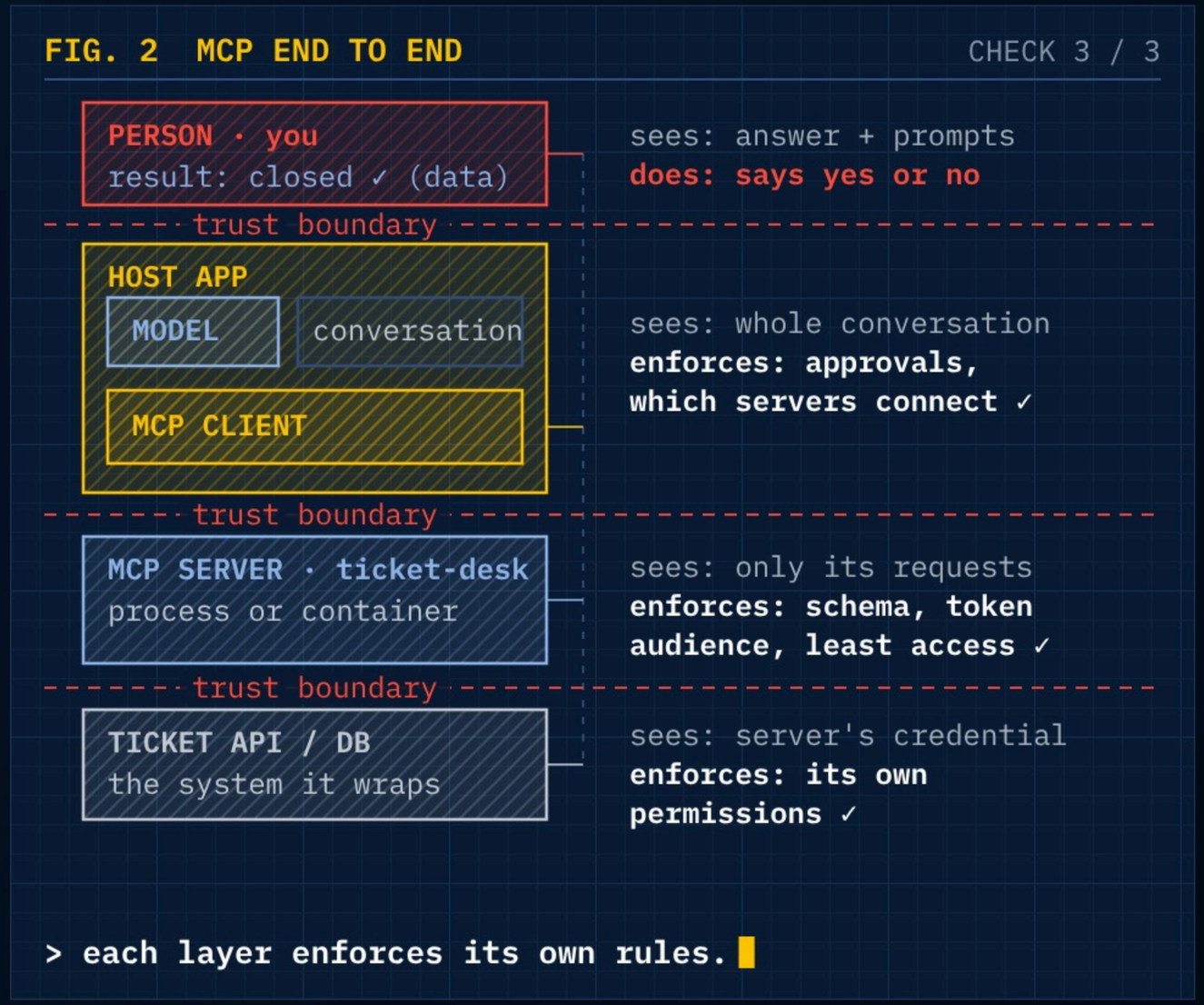
Approvals and which servers connect live in the host

### The server checks everything

Schema, token audience, least access, on every call

### The backend keeps its own rules

The server uses its own credential; backend permissions still apply



Kinvecta

SERVER PRIMITIVES

# Three things a server can offer

### Tools · the model decides

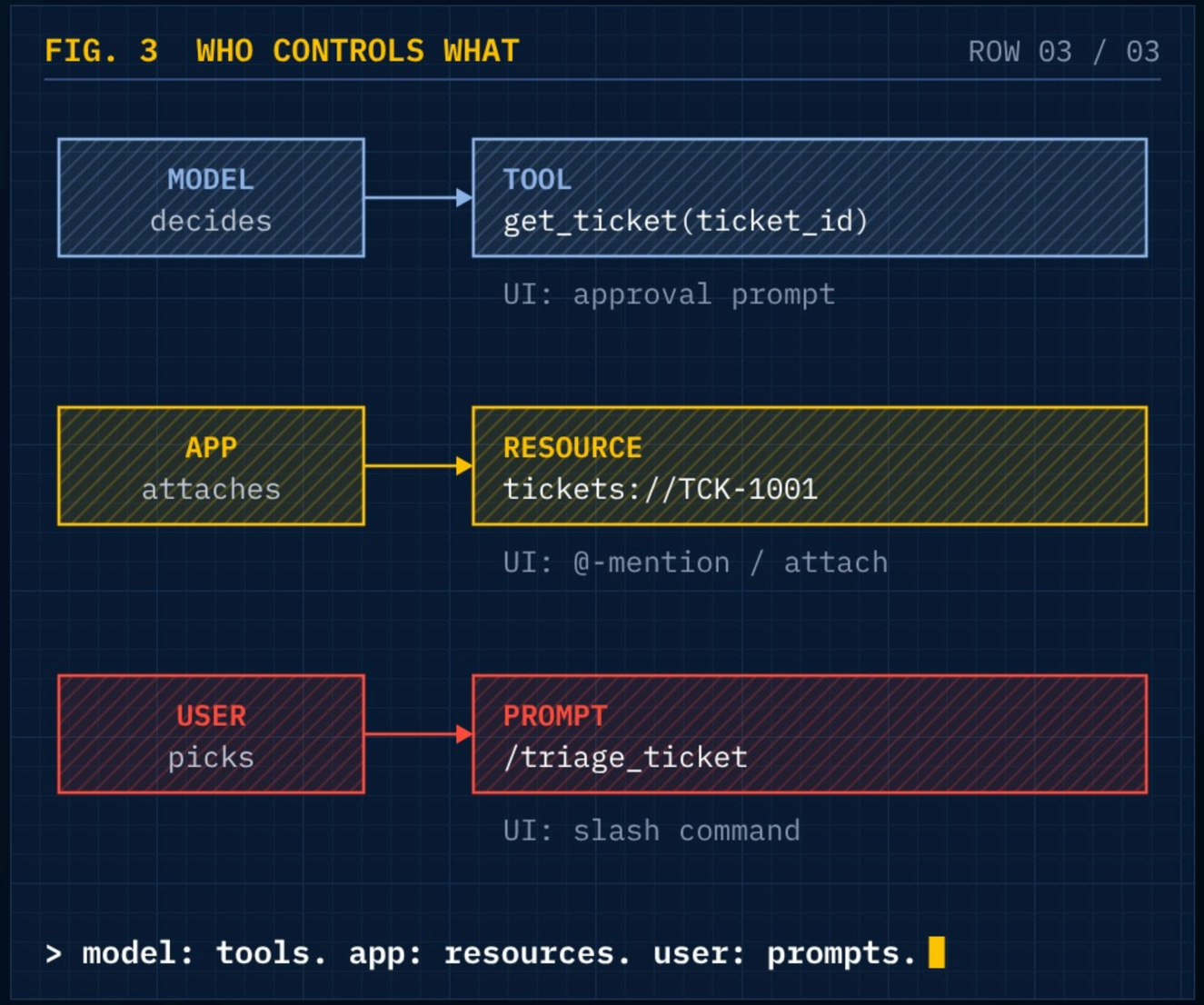
Actions the model can ask to run, like get_ticket

### Resources · the app attaches

Read-only data by URI, like tickets://TCK-1001

### Prompts · the user picks

Templates, usually a slash command like /triage_ticket



Kinvecta

TOOL DEFINITIONS

# Every field has a reader, and only one of them enforces

### The model reads the words

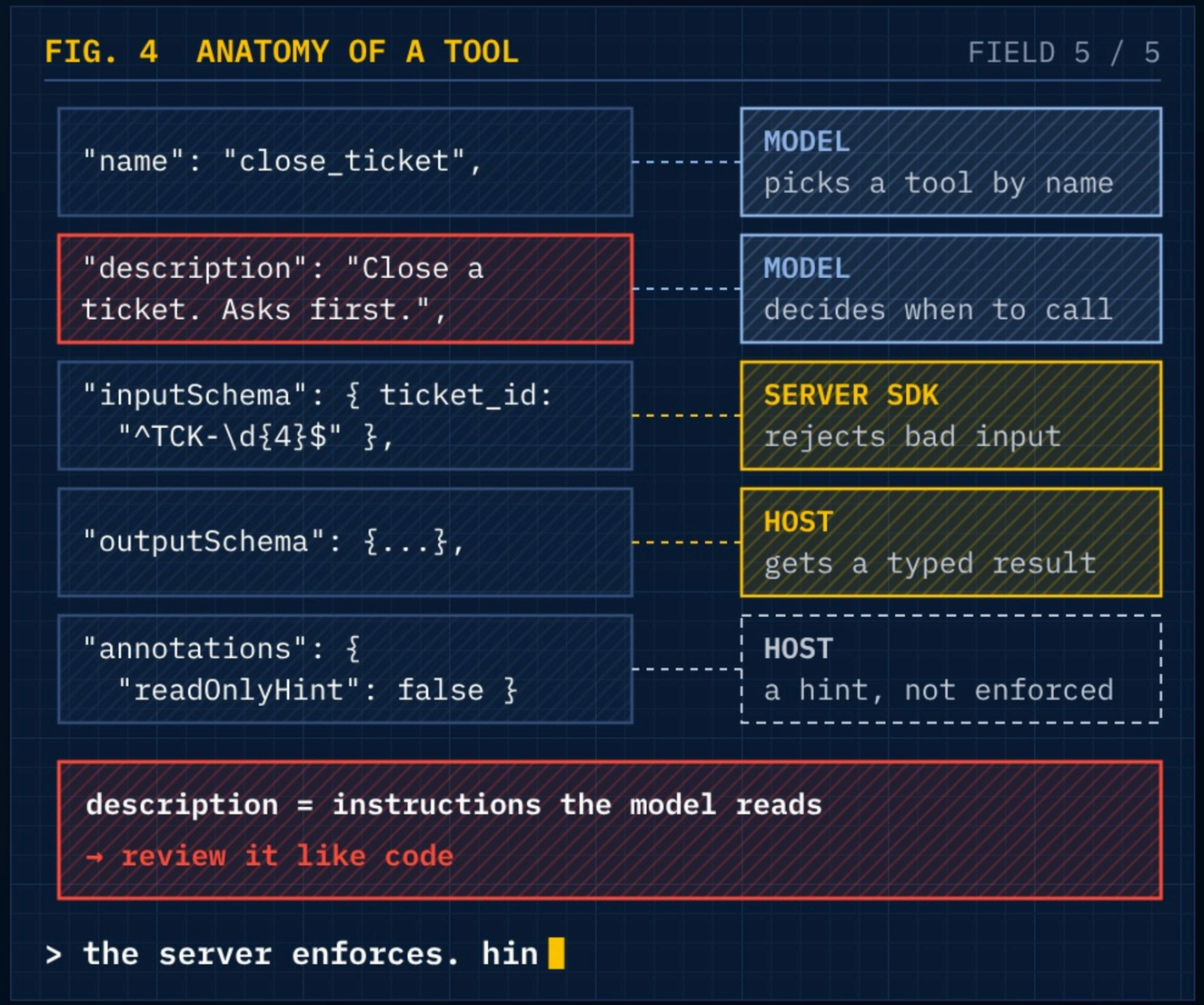
Name and description decide when the tool is called

### The SDK checks the schema

Bad input is rejected before your code runs

### Annotations are only hints

Hosts use them for UI; enforcement is the server's job



Kinvecta

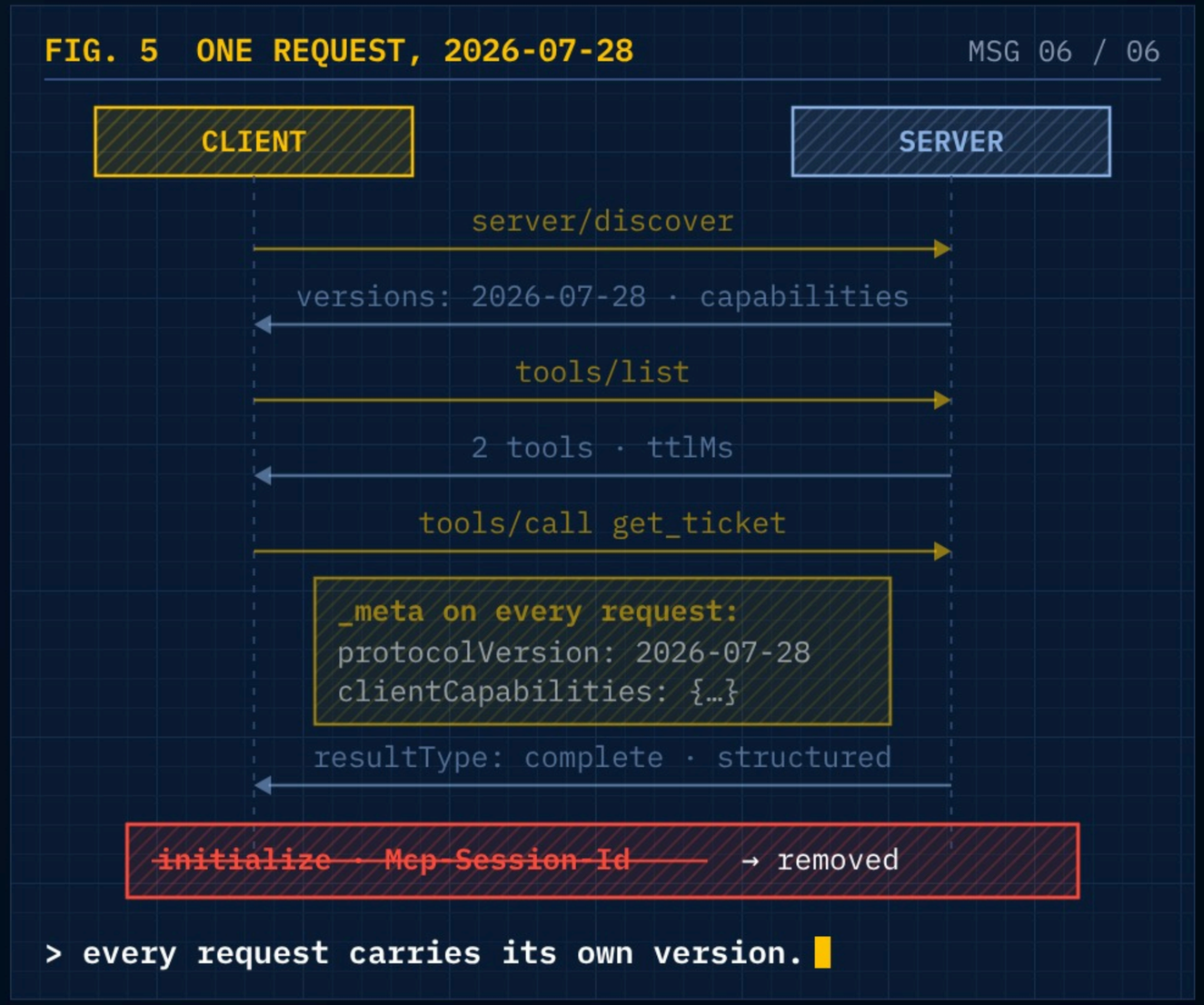
ON THE WIRE

# What one request looks like today

### JSON-RPC 2.0
Every message is a request, a response or a notification

### No handshake, no session
Each request carries its own version and capabilities

### server/discover
Servers must implement it; clients may call it first



Kinvecta

BUILD · SET UP

# Set up the project, on the current SDKs

**terminal · Python**

```
# Python (SDK mcp 2.3.0)
uv init ticket-desk && cd ticket-desk
uv add "mcp[cli]"
uv add --dev pytest inline-snapshot
```

**terminal · TypeScript**

```
# TypeScript (@modelcontextprotocol/server 2.3.1)
npm init -y && npm pkg set type=module
npm install @modelcontextprotocol/server zod
npm install -D tsx typescript @types/node
```

PY · V2 RENAME

### FastMCP → MCPServer

from mcp.server import MCPServer; transport options move to run()

TS · V2 RENAME

### server.tool() → registerTool()

serveStdio(factory), createMcpHandler(factory); ESM only, Zod v4

INSPECTOR 2.11.0

### Needs Node 22.19 or newer

npx @modelcontextprotocol/inspector

Kinvecta

BUILD · PYTHON

# A read-only tool in 16 lines

server.py (excerpt)

```python
from mcp.server import MCPServer
from mcp.types import ToolAnnotations

mcp = MCPServer("ticket-desk", version="1.0.0")
TicketId = Annotated[str, Field(pattern=r"^TCK-\d{4}$")]

@mcp.tool(title="Look up a ticket",
          annotations=ToolAnnotations(read_only_hint=True))
def get_ticket(ticket_id: TicketId) -> Ticket:
    """Return one help-desk ticket by ID. Read-only."""
    log.info("get_ticket %s", ticket_id)   # stderr
    if ticket_id not in TICKETS:
        raise ToolError(f"No ticket {ticket_id}.")
    return Ticket(**TICKETS[ticket_id])

mcp.run()   # stdio by default
```

L05 · L09

### Type hints are the schema

The pattern rejects “1001; DROP TABLE” before your code runs

L13

### ToolError = an error the model reads

It comes back as isError so the model can correct itself

L11

### Logs go to stderr

On stdio, stdout is the wire: one print() breaks it

Kinvecta

BUILD · PYTHON

# An action that asks a person first

server.py (excerpt)

```
01  async def confirm_close(ticket_id: str) -> Elicit[Confirm]:
02      return Elicit(f"Close ticket {ticket_id}?", Confirm)
03
04  @mcp.tool(annotations=ToolAnnotations(read_only_hint=False))
05  async def close_ticket(
06      ticket_id: TicketId,
07      resolution: Annotated[str, Field(min_length=5)],
08      confirm: Annotated[ElicitationResult[Confirm],
09                         Resolve(confirm_close)],
10  ) -> str:
11      match confirm:
12          case AcceptedElicitation(data=Confirm(confirm=True)):
13              TICKETS[ticket_id]["status"] = "closed"
14              return f"{ticket_id} closed."
15          case _:
16              return f"{ticket_id} NOT closed."
```

L08–L09

### Resolve hides confirm

It isn't in the model's schema, so the model can't pre-fill "yes"

L15–L16

### No or cancel: nothing changes

Anything but an accepted "yes" returns NOT closed

ALSO IN THE FILE

### A resource and a prompt

@mcp.resource("tickets://{ticket_id}") and @mcp.prompt()

Kinvecta

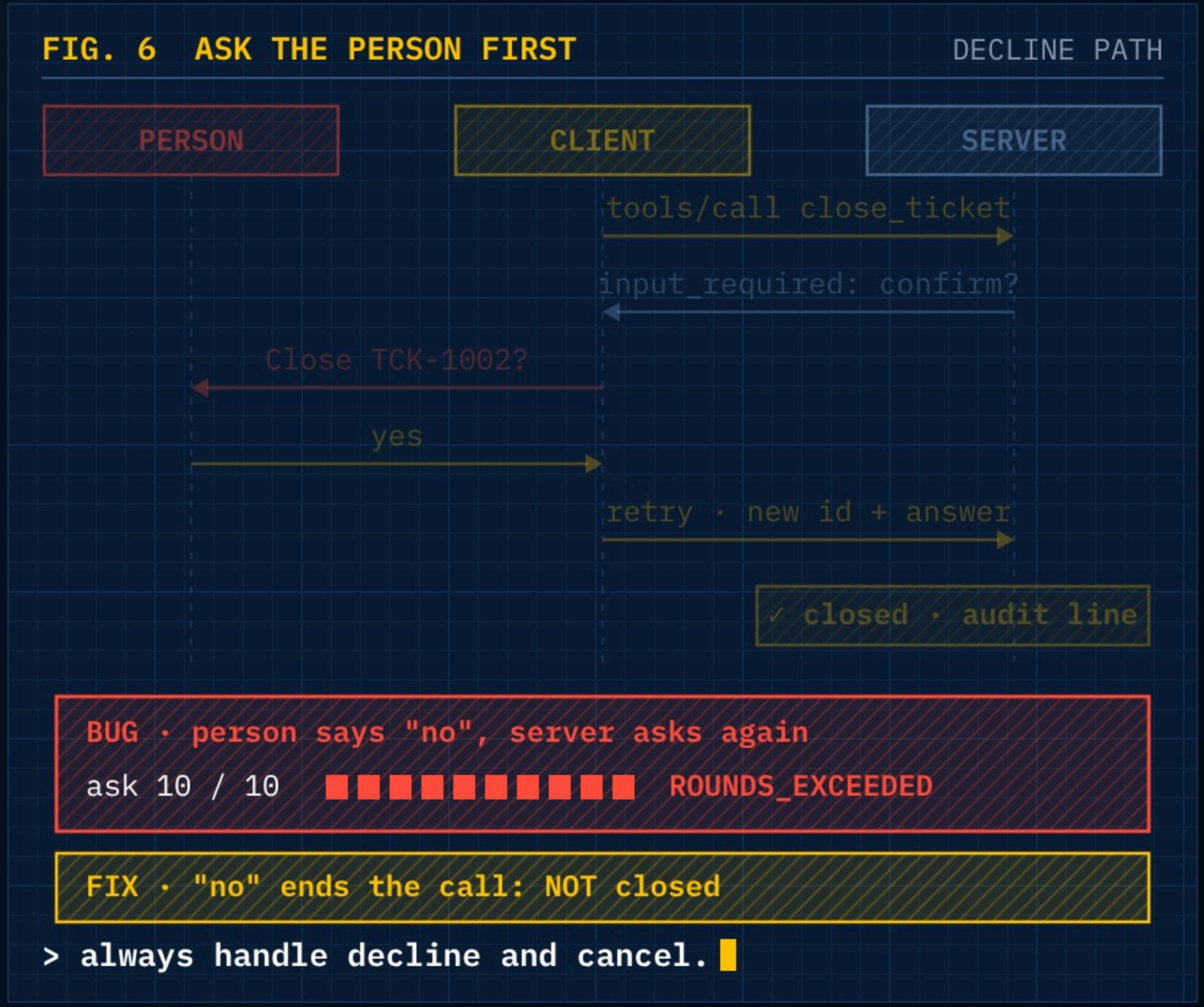
MULTI-ROUND-TRIP REQUESTS

# How the server asks the person

### The server returns input_required
It never sends its own request to the client

### The client asks, then retries
New request id, with the person's answer attached

### Handle no and cancel
Or the server asks again until the client gives up



BUILD · TYPESCRIPT

# The same action, with the decline fix

src/server.ts (excerpt)

```
01  server.registerTool('close_ticket', spec,
02    async ({ ticket_id }, ctx) => {
03      const r = ctx.mcpReq.inputResponses;
04      const view = inputResponse(r, 'confirm');
05      if (view.kind === 'elicit' && view.action !== 'accept')
06        return say(`${ticket_id} NOT closed.`);
07      const answer = acceptedContent(r, 'confirm', Confirm);
08      if (answer === undefined)
09        return inputRequired({ inputRequests: { confirm:
10          inputRequired.elicit({ message: `Close ${ticket_id}?`,
11                                 requestedSchema: Confirm }) } });
12      TICKETS[ticket_id].status = 'closed';
13      console.error(`AUDIT close ${ticket_id}`);  // stderr
14      return say(`${ticket_id} closed.`);
15    });
16  // spec = schema + annotations; say(t) = a text result
```

L04–L06

### The fix: check no and cancel first

acceptedContent() is undefined for missing, no and cancel alike

L08–L11

### No answer yet: return input_required

The client asks the person and retries

L13

### console.error, never console.log

stdout is the wire on stdio

TRANSPORTS

# Two ways to connect

### stdio · local

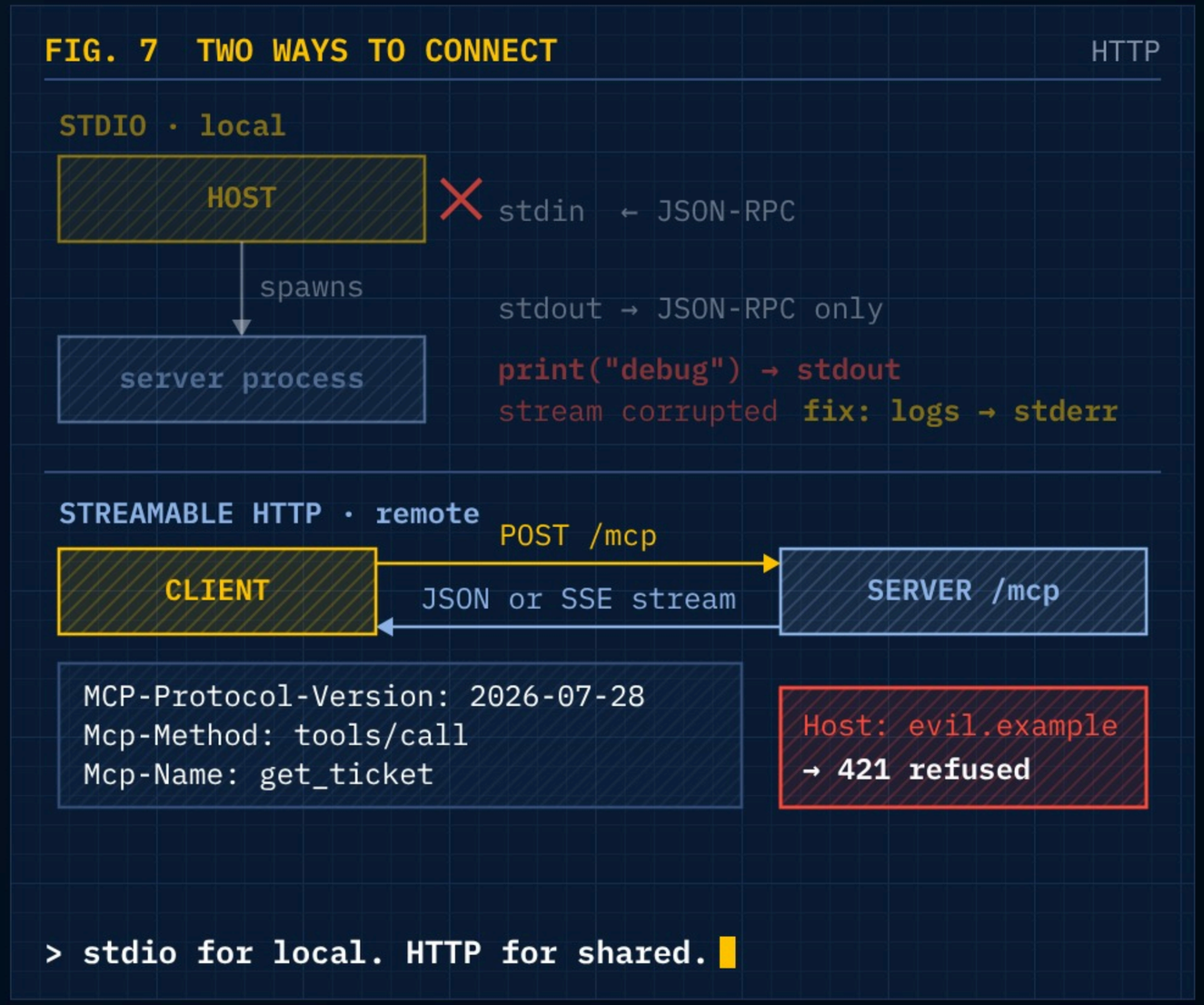
The host starts your server; stdout carries only protocol messages

### Streamable HTTP · shared

One POST /mcp endpoint; required MCP headers on every call

### Guard the HTTP door

Bind 127.0.0.1 locally; allow-list hosts when deployed



Kinvecta

WHERE IT RUNS

# Three places a server can run

### Local, for one person

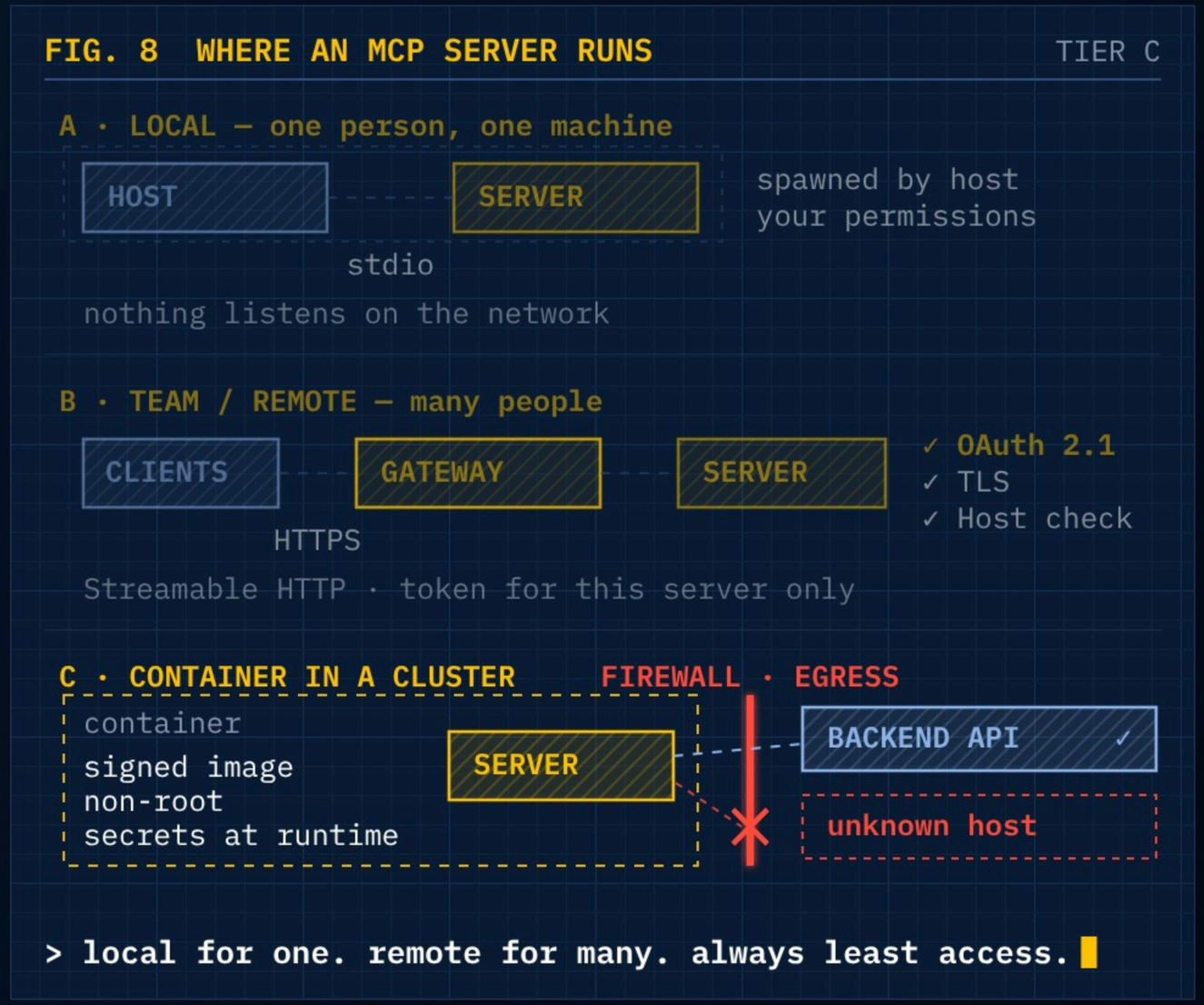
stdio, your permissions, no open port

### Remote, for a team

Streamable HTTP behind a gateway; OAuth 2.1, TLS, host check; a token for this server only

### Container, least access

Signed non-root image, secrets at run time, a firewall that allows only the backend API



TESTING

# Build, inspect, break, fix

### MCP Inspector

Web, CLI and terminal modes; test both protocol eras

### In-memory tests

Call the server in-process: no port, no subprocess

### Snapshot tools/list

A changed description fails the build until reviewed

FIG. 9 BUILD, INSPECT, BREAK, FIX — STEP 06 / 06

| | |
|---|---|
| RUN | $ uv run server.py<br>waits on stdin ✓ |
| INSPECT | $ inspector --cli … tools/list<br>2 tools listed ✓ |
| BREAK | $ get_ticket("1001; DROP TABLE")<br>isError: rejected by schema ✓ |
| TEST | $ pytest: user says "no"<br>ticket still open ✓ |
| SNAPSHOT | $ tools/list → tools.json<br>committed ✓ |
| CI DIFF | $ description changed<br>✗ build fails → review |

> a tool description change is a code change.

Kinvecta

RUN · CONNECT A HOST

# Connect it to a host

**claude_desktop_config.json**

```
{
  "mcpServers": {
    "ticket-desk": {
      "command": "/ABSOLUTE/PATH/TO/uv",
      "args": ["--directory", "/ABS/ticket-desk",
               "run", "server.py"],
      "env": { "TICKET_API_TOKEN": "…" }
    }
  }
}
```

**Claude Code**

```
claude mcp add ticket-desk -- \
  uv --directory /abs/ticket-desk run server.py
claude mcp add --transport http ticket-desk \
  https://mcp.example.com/mcp
```

**VS Code · .vscode/mcp.json**

```
"servers": { "ticket-desk": {
  "type": "stdio", "command": "uv",
  "args": ["run", "server.py"],
  "sandboxEnabled": true } }
```

Kinvecta

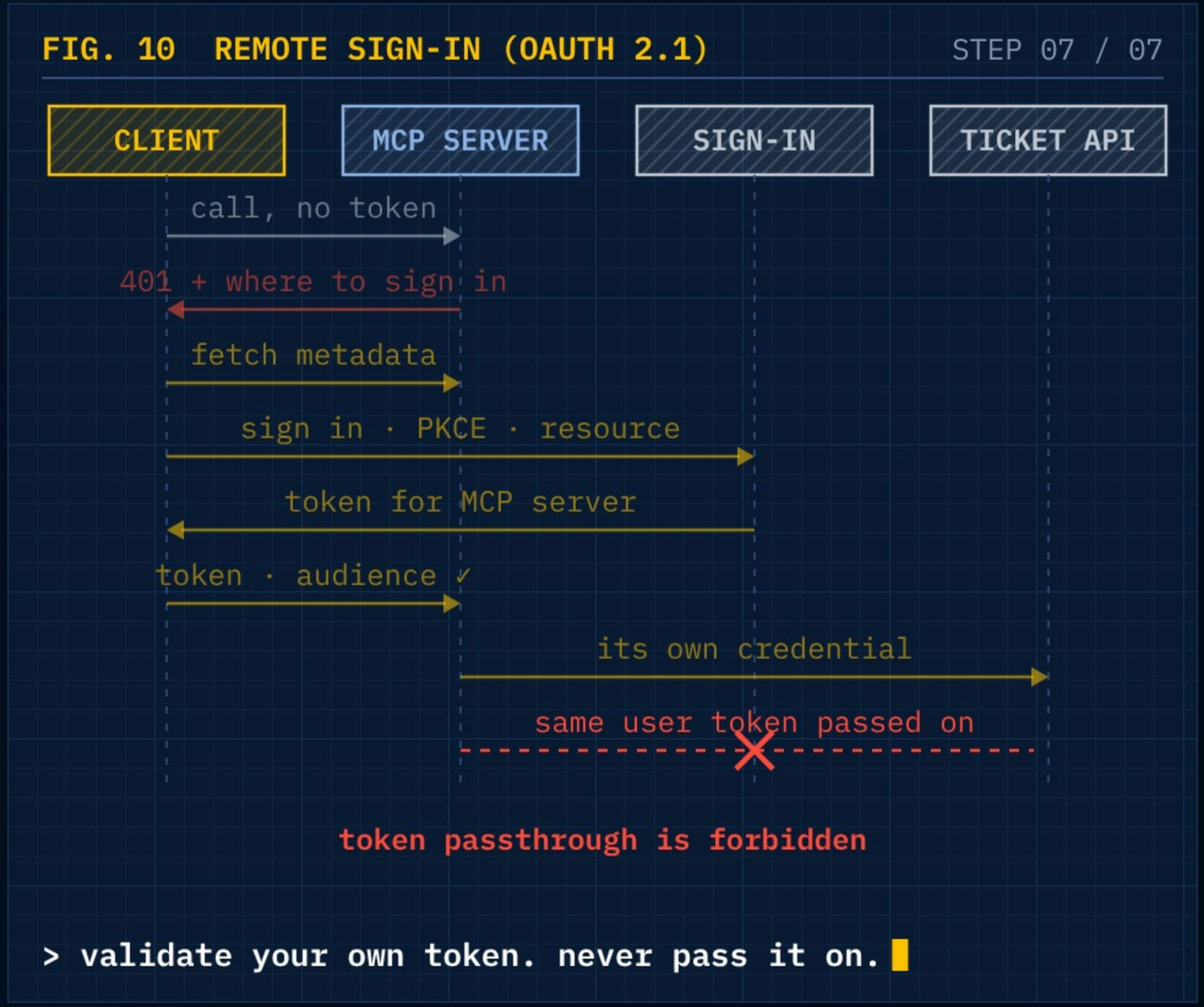
REMOTE SIGN-IN

# OAuth 2.1, and why no passthrough

**Your server is a resource server**
401 points the client to where to sign in

**Tokens are for this server only**
Check the audience on every call; 403 for missing scope

**Never pass the token on**
Call downstream APIs with the server's own credential



Kinvecta

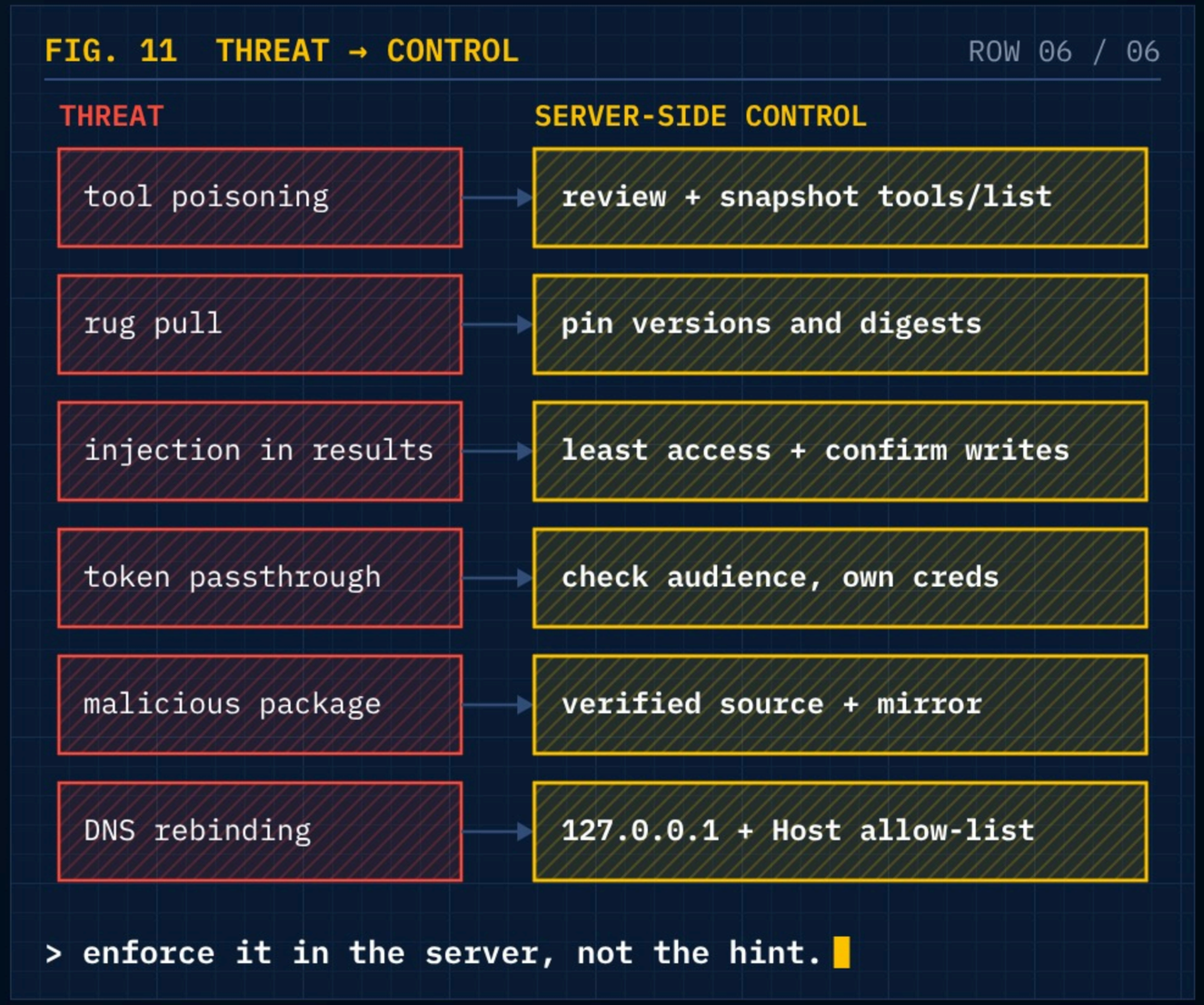
SECURITY

# Every threat has a server-side control

**Annotations are hints**
readOnlyHint helps the UI; enforcement is your code

**The incidents are real**
postmark-mcp, GitHub “toxic agent flow”, Asana data leak

**Map it to OWASP**
OWASP MCP Top 10 (in beta) and the spec’s best practices



Kinvecta

DEVSECOPS

# Security at every step of the server's life

## Design and build

Few tools, tight schemas, a threat model first, every input validated

## Test and secure

Unit tests, Inspector, the decline path; token audience checked, never passed on

## Ship and operate

Container, scan, sign, tool-list snapshot gate; logs, audit, rate limits, alerts on tool changes



Kinvecta

THE RULE OF THUMB

**Few** tools, **tight** schemas.
A person **confirms** every action.
**Validate** your own token. Never pass it on.

The model only sees your descriptions and results, so the server is where safety is enforced.

Kinvecta

HANDS-ON LAB

# Build it, inspect it, break it, fix it

Pause & try

| | | |
|---|---|---|
| A | **Build** | get_ticket and close_ticket in Python or TypeScript, from the starter in the lab folder |
| B | **Inspect** | Inspector CLI: list the tools, call TCK-1001, then call a bad ID |
| C | **Break** | Pairs: poison a description; return injected text from get_ticket; answer “no” to close |
| D | **Fix** | Handle no and cancel; snapshot tools/list; show the diff catching the poisoned description |

Kinvecta

CHECK FOR UNDERSTANDING

# Can every learner answer these?

**1**

With no initialize, how does a server learn the client's protocol version?

**2**

Your stdio server works by hand but the host says "failed". Why?

**3**

destructiveHint is false. Can the host auto-approve the tool?

**4**

Why must a server never forward the token it received?

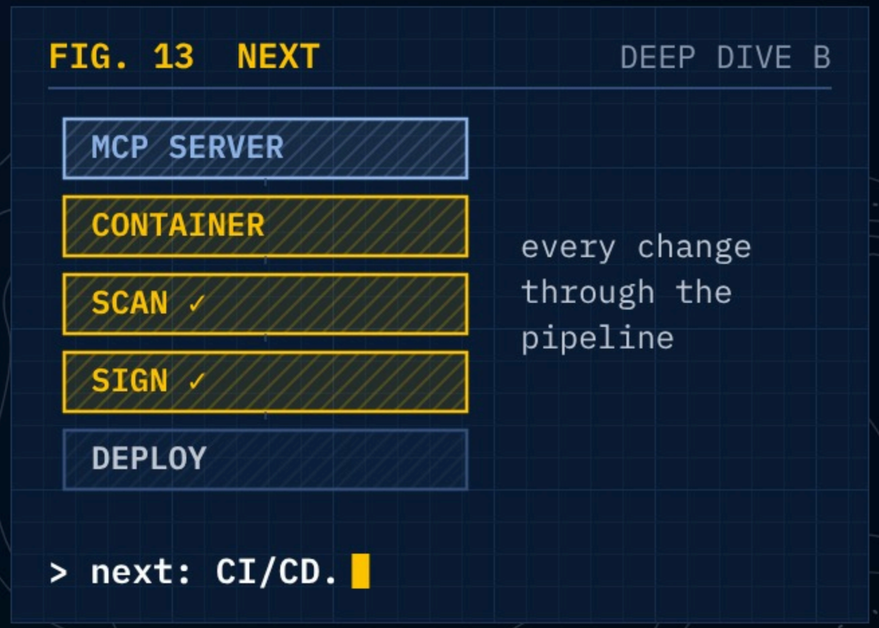
**5**

How does a server get a person's confirmation mid-call?

**6**

Name two controls against a malicious or changed server.

# How does every change reach production safely?




What Deep Dive B covers

- Images and registries
- Scanning tools
- Signing and SBOMs
- The CI/CD pipeline